

CS-114 Fundamentals of Programming

FUNCTIONS STORAGE / SCOPE Passing Variables

**Department of Computer & Software Engineering (DC&SE)
College of E&ME
NUST**

Functions

□ Functions

- Modularize a program
- Software reusability
 - Call function multiple times

□ Local variables

- Known only in the function in which they are defined
- All variables declared in function definitions are local variables

□ Parameters

- Local variables passed to function when called
- Provide outside information

Function Definitions

□ Function prototype

- Tells compiler argument type and return type of function
- **int square(int);**
 - Function takes an **int** and returns an **int**

□ Calling/invoking a function

- **square(x) ;**
- Parentheses an operator used to call function
 - Pass argument x
 - Function gets its own copy of arguments
- After finished, passes back result

Function Definitions

□ Format for function definition

```
return-value-type function-name ( parameter-list )  
{  
    declarations and statements  
}
```

– Parameter list

- Comma separated list of arguments
 - Data type needed for each argument
- If no arguments, use **void** or leave blank

– Return-value-type

- Data type of result returned (use **void** if nothing returned)

Function Definitions

□ Example function

```
int square( int y )  
{  
    return y * y;  
}
```

□ **return** keyword

- Returns data, and control goes to function's caller
 - If no data to return, use **return**;
- Function ends when reaches right brace
 - Control goes to caller

□ Functions cannot be defined inside other functions

□ Next: program examples

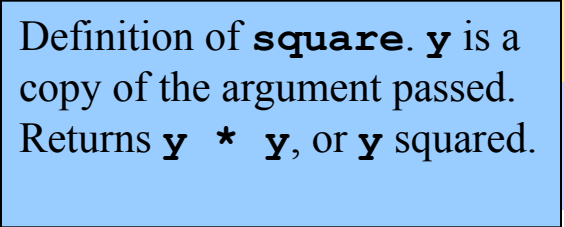
```
1
2
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  int square( int );    // function prototype
9
10 int main()
11 {
12     // loop 10 times and calculate and output
13     // square of x each time
14     for ( int x = 1; x <= 10; x++ )
15         cout << square( x ) << " ";    // function call
16
17     cout << endl;
18
19     return 0;    // indicates successful termination
20
21 } // end main
22
```

Function prototype: specifies data types of arguments and return values. **square** expects an **int**, and returns an **int**.

Parentheses () cause function to be called. When done, it returns the result.

```
23 // square function definition returns square of an integer
24 int square( int y ) // y is a copy of argument to function
25 {
26     return y * y; // returns square of y as an int
27
28 } // end function square
```

```
1  4  9 16 25 36 49 64 81 100
```



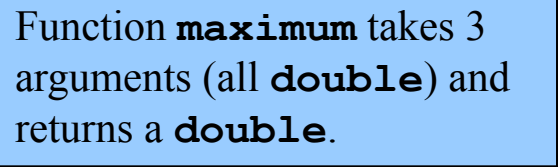
Definition of **square**. **y** is a copy of the argument passed. Returns **y * y**, or **y** squared.

```
1
2
3  #include <iostream>
4
5  Using namespace std;
6
7
8  void interchange( int , int );    // function prototype
9
10 int main()
11 {
12     int num1, num2;
13
14     cout << "Enter two numbers";
15     cin >> num1 >> num2;
16
17     interchange( num1, num2 );    // function call
18
19     cout << endl;
20
21     return 0;    // indicates successful termination
22 }
```



```
23 // Interchange function definition
24 void interchange( int x, int y ) // x and y are copy of arguments to
    function
25 {
26     int c;
27     c = x;
28     x = y;
29     y = c;
30 cout<<"Interchanged values are:<<x<<" "<<<y;
    } // end function interchange
```

```
1
2 // Finding the maximum of three floating-point numbers.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 double maximum( double, double, double ); // function prototype
10
11 int main()
12 {
13     double number1;
14     double number2;
15     double number3;
16
17     cout << "Enter three floating-point numbers: ";
18     cin >> number1 >> number2 >> number3;
19
20     // number1, number2 and number3 are arguments to
21     // the maximum function call
22     cout << "Maximum is: "
23         << maximum( number1, number2, number3 ) << endl;
24
25     return 0; // indicates successful termination
```



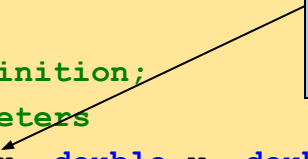
Function **maximum** takes 3 arguments (all **double**) and returns a **double**.

```

26
27 } // end main
28
29 // function maximum definition;
30 // x, y and z are parameters
31 double maximum( double x, double y, double z )
32 {
33     double max = x;    // assume x is largest
34
35     if ( y > max )      // if y is larger,
36         max = y;        // assign y to max
37
38     if ( z > max )      // if z is larger,
39         max = z;        // assign z to max
40
41     return max;         // max is largest value
42
43 } // end function maximum

```

Comma separated list for multiple parameters.



Enter three floating-point numbers: 99.32 37.3 27.1928

Maximum is: 99.32

Enter three floating-point numbers: 1.1 3.333 2.22

Maximum is: 3.333

Enter three floating-point numbers: 27.9 14.31 88.99

Maximum is: 88.99

Storage Classes

□ Variables have attributes

- Have seen name, type, size, value
- Storage class
 - How long variable exists in memory
- Scope
 - Where variable can be referenced in program
- Linkage
 - For multiple-file program

Storage Classes

□ Automatic storage class

- Variable created when program enters its block
- Variable destroyed when program leaves block
- Only local variables of functions can be automatic
 - Automatic by default
 - keyword **auto** explicitly declares automatic
- **register** keyword
 - Hint to place variable in high-speed register
 - Good for often-used items (loop counters)
 - Often unnecessary, compiler optimizes
- Specify either **register** or **auto**, not both
 - **register int counter = 1;**

Storage Classes

□ Static storage class

- Variables exist for entire program
- May not be accessible, scope rules still apply (more later)

□ **static** keyword

- Local variables in function
- Keeps value between function calls
- Only known in own function

□ **extern** keyword

- Default for global variables/functions
 - Globals: defined outside of a function block
- Known in any function that comes after it

Scope Rules

□ Scope

- Portion of program where identifier can be used

□ File scope

- Defined outside a function, known in all functions
- Global variables, function definitions and prototypes

□ Function scope

- Can only be referenced inside defining function

Scope Rules

□ Block scope

- Begins at declaration, ends at right brace }
- Can only be referenced in this range
- Local variables, function parameters
- **static** variables still have block scope
- Storage duration does not affect scope

□ Function-prototype scope

- Parameter list of prototype
- Names in prototype optional
- Compiler ignores


```

1
2 // A scoping example.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 void useLocal( void );
9 void useStaticLocal( void );
10 void useGlobal( void );
11
12 int x = 1; // global variable
13
14 int main()
15 {
16     int x = 5; // local variable to main
17
18     cout << "local x in main's outer scope is " << x << endl;
19
20     { // start new scope
21
22         int x = 7;
23
24         cout << "local x in main's inner scope is " << x << endl;
25
26     } // end new scope

```

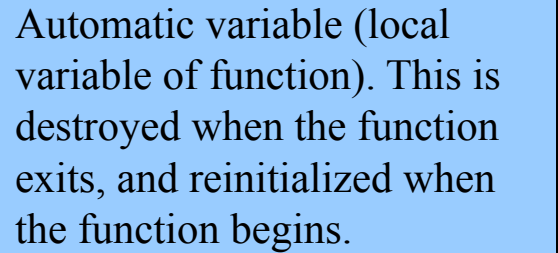
Declared outside of function;
global variable with file
scope.

Local variable with function
scope.

Create a new block, giving **x**
block scope. When the block
ends, this **x** is destroyed.

```
27
28     cout << "local x in main's outer scope is " << x << endl;
29
30     useLocal();           // useLocal has local x
31     useStaticLocal();    // useStaticLocal has static local x
32     useGlobal();         // useGlobal uses global x
33     useLocal();           // useLocal reinitializes its local x
34     useStaticLocal();    // static local x retains its prior value
35     useGlobal();         // global x also retains its value
36
37     cout << "\nlocal x in main is " << x << endl;
38
39     return 0;           // indicates successful termination
40
41 } // end main
42
```

```
43 // useLocal reinitializes local variable x during each call
44 void useLocal( void )
45 {
46     int x = 25; // initialized each time useLocal is called
47
48     cout << endl << "local x is " << x << endl;
49     cout << " on entering useLocal\n";
50     ++x;
51     cout << "local x is " << x << endl;
52     cout << " on exiting useLocal\n";
53
54 } // end function useLocal
55
```



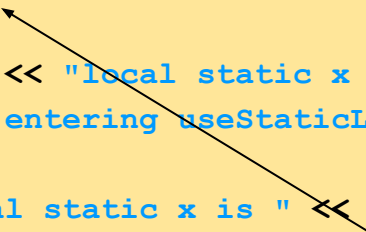
Automatic variable (local variable of function). This is destroyed when the function exits, and reinitialized when the function begins.

```

56 // useStaticLocal initializes static local variable x only the
57 // first time the function is called; value of x is saved
58 // between calls to this function
59 void useStaticLocal( void )
60 {
61     // initialized only first time useStaticLocal is called
62     static int x = 50;
63
64     cout << endl << "local static x is " << x
65         << " on entering useStaticLocal" << endl;
66     ++x;
67     cout << "local static x is " <<
68         << " on exiting useStaticLo
69
70 } // end function useStaticLocal
71

```

Static local variable of function; it is initialized only once, and retains its value between function calls.



```

72 // useGlobal modifies global variable x during each call
73 void useGlobal( void )
74 {
75     cout << endl << "global x is " << x
76         << " on entering useGlobal" << endl;
77     x *= 10;
78     cout << "global x is " << x
79         << " on exiting useGlobal" << endl;
80
81 } // end function useGlobal

```

This function does not declare any variables. It uses the global **x** declared in the beginning of the program.

```

local x in main's outer scope is 5
local x in main's inner scope is 7
local x in main's outer scope is 5

```

```

local x is 25 on entering useLocal
local x is 26 on exiting useLocal

```

```

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

```

```

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal

```

local x is 25 on entering useLocal

local x is 26 on exiting useLocal

local static x is 51 on entering useStaticLocal

local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal

global x is 100 on exiting useGlobal

local x in main is 5

Unary Scope Resolution Operator

- Unary scope resolution operator (`::`)
 - Access global variable if local variable has same name
 - Not needed if names are different
 - Use `::variable`
 - `y = ::x + 3;`
 - Good to avoid using same names for locals and globals

References and Reference Parameters

□ Call by value

- Copy of data passed to function
- Changes to copy do not change original
- Prevent unwanted side effects

□ Call by reference

- Function can directly access data
- Changes affect original

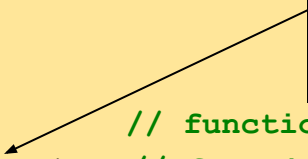
Reference Parameters

□ Reference parameter

- Alias for argument in function call
 - Passes parameter by reference
- Use **&** after data type in prototype
 - **void myFunction(int &data)**
 - Read “**data** is a reference to an **int**”
- Function call format the same
 - However, original can now be changed

```
1 // Fig. 3.20: fig03_20.cpp
2 // Comparing pass-by-value and pass-by-reference
3 // with references.
4 #include <iostream>
5
6 using std::cout;
7 using std::endl;
8
9 int squareByValue( int );           // function prototype
10 void squareByReference( int & );    // function prototype
11
12 int main()
13 {
14     int x = 2;
15     int z = 4;
16
17     // demonstrate squareByValue
18     cout << "x = " << x << " before squareByValue\n";
19     cout << "Value returned by squareByValue: "
20         << squareByValue( x ) << endl;
21     cout << "x = " << x << " after squareByValue\n" << endl;
22
```

Notice the & operator,
indicating pass-by-reference.



```

23 // demonstrate squareByReference
24 cout << "z = " << z << " before squareByReference" << endl;
25 squareByReference( z );
26 cout << "z = " << z << " after squareByReference" << endl;
27
28 return 0; // indicates successful termination
29 } // end main
30
31 // squareByValue multiplies number by itself
32 // result in number and returns the new value
33 int squareByValue( int number )
34 {
35     return number *= number; // caller's argument not modified
36 }
37 // end function squareByValue
38
39 // squareByReference multiplies numberRef by itself
40 // stores the result in the variable to which numberRef
41 // refers in function main
42 void squareByReference( int &numberRef )
43 {
44     numberRef *= numberRef; // caller's argument modified
45 }
46 // end function squareByReference

```

Changes **number**, but original parameter (**x**) is not modified.

Changes **numberRef**, an alias for the original parameter. Thus, **z** is changed.

```
x = 2 before squareByValue  
Value returned by squareByValue: 4  
x = 2 after squareByValue
```

```
z = 4 before squareByReference  
z = 16 after squareByReference
```

References and Reference Parameters

References as aliases to other variables

- Refer to same variable
- Can be used within a function

```
int count = 1; // declare integer variable count
Int &cRef = count; // create cRef as an alias for count
++cRef; // increment count (using its alias)
```

□ References must be initialized when declared

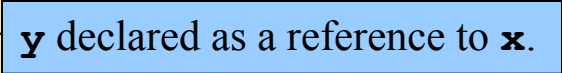
- Otherwise, compiler error
- Dangling reference
 - Reference to undefined variable

```

1 // Fig. 3.21: fig03_21.cpp
2 // References must be initialized.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 int main()
9 {
10     int x = 3;
11
12     // y refers to (is an alias for) x
13     int &y = x;
14
15     cout << "x = " << x << endl << "y = " << y << endl;
16     y = 7;
17     cout << "x = " << x << endl << "y = " << y << endl;
18
19     return 0; // indicates successful termination
20
21 } // end main

```

y declared as a reference to x.



```

x = 3
y = 3
x = 7
y = 7

```

```
1 // Fig. 3.22: fig03_22.cpp
2 // References must be initialized.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 int main()
9 {
10     int x = 3;
11     int &y; // Error: y must be initialized
12
13     cout << "x = " << x << endl << "y = " << y << endl;
14     y = 7;
15     cout << "x = " << x << endl << "y = " << y << endl;
16
17     return 0; // indicates successful termination
18
19 } // end main
```

Uninitialized reference –
compiler error.

'y' : references must be initialized

Default Arguments

□ Function call with omitted parameters

- If not enough parameters, rightmost go to their defaults
- Default values
 - Can be constants, global variables, or function calls

□ Set defaults in function prototype

```
int myFunction( int x = 1, int y = 2, int z = 3 );
```

- **myFunction(3)**
 - **x = 3**, **y** and **z** get defaults (rightmost)
- **myFunction(3, 5)**
 - **x = 3**, **y = 5** and **z** gets default


```

1 // Fig. 3.23: fig03_23.cpp
2 // Using default arguments.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function prototype that specifies default arguments
9 int boxVolume( int length = 1, int width = 1, int height = 1 );
10
11 int main()
12 {
13     // no arguments--use default values for all dimensions
14     cout << "The default box volume is: " << boxVolume();
15
16     // specify length; default width and height
17     cout << "\n\nThe volume of a box with length 10,\n"
18         << "width 1 and height 1 is: " << boxVolume( 10 );
19
20     // specify length and width; default height
21     cout << "\n\nThe volume of a box with length 10,\n"
22         << "width 5 and height 1 is: " << boxVolume( 10, 5 );
23

```

Set defaults in function prototype.

Function calls with some parameters missing – the rightmost parameters get their defaults.

```

24 // specify all arguments
25 cout << "\n\nThe volume of a box with length 10,\n"
26     << "width 5 and height 2 is: " << boxVolume( 10, 5, 2 )
27     << endl;
28
29 return 0; // indicates successful termination
30
31 } // end main
32
33 // function boxVolume calculates the volume of a box
34 int boxVolume( int length, int width, int height )
35 {
36     return length * width * height;
37
38 } // end function boxVolume

```

The default box volume is: 1

The volume of a box with length 10,
width 1 and height 1 is: 10

The volume of a box with length 10,
width 5 and height 1 is: 50

The volume of a box with length 10,
width 5 and height 2 is: 100

Function Overloading

□ Function overloading

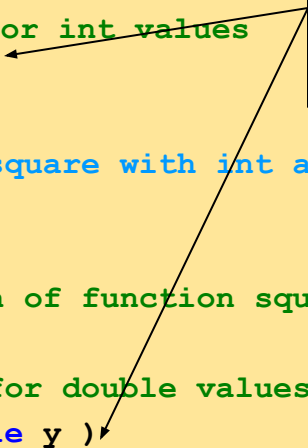
- Functions with same name and different parameters
 - Should perform similar tasks
 - I.e., function to square **ints** and function to square **floats**
- ```
int square(int x) {return x * x;}
float square(float x) { return x * x; }
```

## □ Overloaded functions distinguished by signature

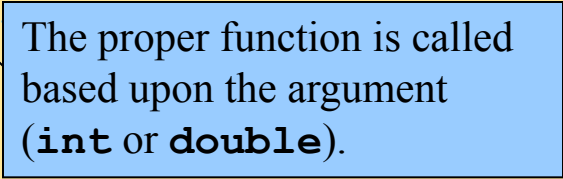
- Based on name and parameter types (order matters)
- Name mangling
  - Encodes function identifier with parameters
- Type-safe linkage
  - Ensures proper overloaded function called

```
1 // Fig. 3.25: fig03_25.cpp
2 // Using overloaded functions.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function square for int values
9 int square(int x)
10 {
11 cout << "Called square with int argument: " << x << endl;
12 return x * x;
13
14 } // end int version of function square
15
16 // function square for double values
17 double square(double y)
18 {
19 cout << "Called square with double argument: " << y << endl;
20 return y * y;
21
22 } // end double version of function square
23
```

Overloaded functions have the same name, but the different parameters distinguish them.

A blue rectangular box with a black border contains the text "Overloaded functions have the same name, but the different parameters distinguish them." Two black arrows originate from the box. One arrow points from the text to the function signature "int square( int x )" on line 9 of the code. The other arrow points from the text to the function signature "double square( double y )" on line 17 of the code.

```
24 int main()
25 {
26 int intResult = square(7); // calls int version
27 double doubleResult = square(7.5); // calls double version
28
29 cout << "\nThe square of integer 7 is 49\n";
30 cout << "\nThe square of double 7.5 is 56.25\n";
31 cout << endl;
32
33 return 0; // indicates successful termination
34
35 } // end main
```



The proper function is called based upon the argument (**int** or **double**).

```
Called square with int argument: 7
Called square with double argument: 7.5

The square of integer 7 is 49
The square of double 7.5 is 56.25
```

# Introduction to Search Algorithms

- A search algorithm is a method of locating a specific item of information in a larger collection of data.

# The Linear Search

- This is a very simple algorithm.
- It uses a loop to sequentially step through an array, starting with the first element.
- It compares each element with the value being searched for and stops when that value is found or the end of the array is reached.

# Searching Arrays: Linear Search

- Search array for a key value
- Linear search
  - Compare each element of array with key value
    - Start at one end, go to other
  - Useful for small and unsorted arrays
    - Inefficient
    - If search key not present, examines every element

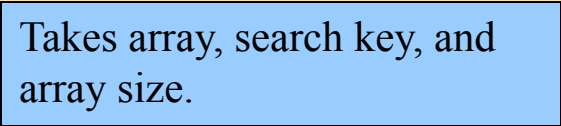


```

1 // Fig. 4.19: fig04_19.cpp
2 // Linear search of an array.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 int linearSearch(const int [], int, int); // prototype
10
11 int main()
12 {
13 const int arraySize = 100; // size of array a
14 int a[arraySize]; // create array a
15 int searchKey; // value to locate in a
16
17 for (int i = 0; i < arraySize; i++) // create some data
18 a[i] = 2 * i;
19
20 cout << "Enter integer search key: ";
21 cin >> searchKey;
22
23 // attempt to locate searchKey in array a
24 int element = linearSearch(a, searchKey, arraySize);
25

```

Takes array, search key, and array size.



```
26 // display results
27 if (element != -1)
28 cout << "Found value in location " << element << endl;
29 else
30 cout << "Value not found" << endl;
31
32 return 0; // indicates successful termination
33
34 } // end main
35
36 // compare key to every element of array until location is
37 // found or until end of array is reached; return subscript of
38 // element if key or -1 if key not found
39 int linearSearch(const int array[], int key, int sizeOfArray)
40 {
41 for (int j = 0; j < sizeOfArray; j++)
42
43 if (array[j] == key) // if found,
44 return j; // return location of key
45
46 return -1; // key not found
47
48 } // end function linearSearch
```

Enter integer search key: 36

Found value in location 18

Enter integer search key: 37

Value not found